MODULE 04 · DOMAIN 1 - KUBERNETES FUNDAMENTALS

The API and the object model

What an object is, how the API is versioned, and how Kubernetes learns new kinds.

DOMAIN 1 of 4 EXAM WEIGHT 44% MODULE 4 of 35

admin@sthithapragna.com

Everything in Kubernetes is **an object**, read and written through **one REST API**.

The anatomy of an object

Trimmed from kubectl get deployment booking-api -o yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: booking-api
  namespace: quayside
  labels: {app: booking-api}
  uid: 6f1c2e9a-4b7d-4e21-9a0c-2d5e8f3b1a77
spec:
  replicas: 3
  # ...the Pod template...
status:
  replicas: 3
  readyReplicas: 2
```

1-2 Which API group and version, and which kind of object.

4-7 Identity. Name is unique per kind in a namespace; uid is unique forever.

8 spec: the desired state. You write this.

11 status: the observed state. The system writes this.

spec versus status

SPEC

What you want

- Written by you, or your tooling
- Declares desired state
- Changes when you change your mind
- Example: replicas 3

STATUS

What is true

- Written by controllers and the kubelet
- Reports observed state
- Changes as the world changes
- Example: readyReplicas 2

THE TIEBREAKER

You write spec. The system writes status. Reconciliation is the work of closing the gap between them.

Which fields are required?

- **apiVersion, kind and metadata** appear on every object. metadata must carry a name.
- **spec** is required for most kinds, such as Pods, Deployments and Services.
- **Not every kind has a spec.** ConfigMaps and Secrets carry data instead.
- **status** is never something you write in a manifest.

> ON THE EXAM
>
> If asked for the fields every manifest needs, the documented answer is apiVersion, kind, metadata and spec.

API groups you will see in manifests

APIVERSION	KINDS
v1	The core group: Pod, Service, ConfigMap, Secret, Namespace, Node, PVC
apps/v1	Deployment, ReplicaSet, StatefulSet, DaemonSet
batch/v1	Job, CronJob
networking.k8s.io/v1	Ingress, IngressClass, NetworkPolicy
rbac.authorization.k8s.io/v1	Role, ClusterRole, RoleBinding, ClusterRoleBinding
autoscaling/v2	HorizontalPodAutoscaler
gateway.networking.k8s.io/v1	Gateway, HTTPRoute. Installed separately, as CRDs

ON THE EXAM

A bare v1 means the core group. Everything else is group/version.

The URLs behind kubectl

Illustrative: what the API server actually receives.

```
$ kubectl get pods -n quayside
  GET /api/v1/namespaces/quayside/pods

$ kubectl get deployment booking-api -n quayside
  GET /apis/apps/v1/namespaces/quayside/deployments/booking-api

$ kubectl create -f nightly-export-job.yaml
  POST /apis/batch/v1/namespaces/quayside/jobs
```

2 The core group lives under /api/v1.

5 Named groups live under /apis/group/version.

8 Create is a POST. Get is a GET. It is plain REST over HTTPS.

Alpha, beta, stable: what each version promises

EXAMPLE	LEVEL	THE PROMISE
v1alpha1	Alpha	Experimental. Off by default. May change or vanish in any release.
v2beta3	Beta	Well tested, but may still change incompatibly. New beta APIs are off by default.
v1	Stable (GA)	Will not be removed within a major version of Kubernetes.

ON THE EXAM

The version name tells you the promise. v2beta3 means: third beta of version 2, still allowed to change.

Currency check: the deprecation policy

- **Stable APIs:** may be deprecated, but never removed within Kubernetes 1.x.
- **Beta APIs:** still served at least 9 months or 3 minor releases after deprecation.
- **Alpha APIs:** can be removed in any release, with no notice.
- The **12 months** often quoted comes from an older rule. Today it applies to kubectl flags.
- Kubernetes ships about three minor releases a year, one every four months or so.

> ON THE EXAM
>
> Calling a deprecated API still works, but the response carries a warning. Treat it as a deadline.

The life of an API version

01

v1alpha1

Experiment. May change freely.

›

02

v1beta1

Tested. Alpha removed.

›

03

v1beta2

v1beta1 deprecated.

›

04

v1

Stable. Betas deprecated.

›

05

Betas removed

After their deprecation window.

ON THE EXAM

Old manifests break on upgrade when they still name a beta version that has since been removed.

Asking the cluster about its own API

COMMAND	ANSWERS
`kubectl api-resources`	Which kinds exist, their short names, group, and whether namespaced
`kubectl api-versions`	Which group/version pairs this cluster serves
`kubectl explain deployment.spec.replicas`	What a field means and its type, from the live schema
`kubectl get deploy booking-api -o yaml`	The full object as stored, spec and status

ON THE EXAM

Field documentation: explain. List of kinds: api-resources. The live object: get -o yaml.

Three ways to manage objects

TECHNIQUE	EXAMPLE	TRADE-OFF
Imperative commands	kubectl scale deploy web --replicas=5	Fast to type. No file, no history, hard to review.
Imperative object configuration	kubectl create -f web.yaml	A file exists, but you say create, replace or delete.
Declarative object configuration	kubectl apply -f manifests/	The files are the truth. Kubernetes works out the changes.

ON THE EXAM

Declarative is the foundation of GitOps: the files in Git are the desired state.

Why reconciliation is so robust

- **Level-triggered.** Controllers compare current state with desired state, not a list of events.
- **Missed events do not matter.** A restarted controller just looks again and acts.
- **Drift gets corrected.** Delete a Deployment's Pod by hand and a new one appears.
- **Hand edits to managed objects are overwritten.** Change the owner, not the child.

> ON THE EXAM
>
> To change what a controller manages, change the controller's spec. Editing its children is undone.

Kubernetes can learn **new kinds of object**. That is how most of the ecosystem plugs in.

Two ways to extend the API

CUSTOMRESOURCEDEFINITION

Declare a new kind

- Apply one YAML object
- API server stores it in etcd
- No extra server to run
- Used by most projects

AGGREGATION LAYER

Run your own API server

- You build and run an extension API server
- kube-apiserver proxies a group to it
- Your own storage and logic
- Example: metrics-server

THE TIEBREAKER

Almost always a CRD. Aggregation when the data is not stored objects, like live metrics from metrics.k8s.io.

A custom resource at Quayside

After a CRD for FerryRoute is installed, this is an ordinary object.

```
1  apiVersion: quayside.io/v1
2  kind: FerryRoute
3  metadata: {name: dover-calais}
4  spec:
5    departurePort: Dover
6    arrivalPort: Calais
7    sailingsPerDay: 12
8
9  $ kubectl get ferryroutes
10 NAME           AGE
11 dover-calais   4m
```

1-2 Quayside's own group and kind, defined by a CRD.

9 kubectl works with it like any built-in kind.

On its own, a CRD only stores the object. Nothing happens until a controller acts on it.

The operator pattern

- **Operator = custom resource + custom controller.** The controller reconciles the resource.
- **It encodes a human operator's know-how:** install, upgrade, back up, fail over.
- **Examples:** Prometheus Operator, cert-manager, CloudNativePG, Rook for Ceph storage.
- You declare a database or a certificate; the operator keeps it running correctly.

> REMEMBER
>
> A CRD adds a noun. A controller adds a verb. Together, they are an operator.

> A vendor's scheduling add-on for Quayside offers its API at three versions: v1alpha2, v1beta1 and v1.
>
> The platform lead asks what, exactly, the v1beta1 version promises before any team relies on it.

What does the v1beta1 version promise?

A. Nothing; it may vanish in any release without notice

B. It is final and will never be changed or removed

C. It is well tested, but may still change incompatibly

D. It can only be used on managed Kubernetes services

◆ PAUSE AND DECIDE

What does the v1beta1 version promise?

A Nothing; it may vanish in any release without notice

That is the alpha promise, not beta.

B It is final and will never be changed or removed

Closer to stable, and even stable APIs can be deprecated.

C It is well tested, but may still change incompatibly

Exactly the beta promise. Production should prefer v1.

D It can only be used on managed Kubernetes services

Version levels have nothing to do with where a cluster runs.

THE SIGNAL

Alpha: may vanish. Beta: may change. Stable: stays within the major version.

> Quayside upgrades an old test cluster to a current Kubernetes release. Re-applying a years-old manifest now fails.
>
> The error says: no matches for kind Ingress in version extensions/v1beta1. Other Ingress manifests apply cleanly.

What is the cause, and the fix?

A Ingress was removed; rewrite everything using the Gateway API

B That beta version was removed; use networking.k8s.io/v1

C The Ingress CRD is missing and needs to be installed

D An alpha feature gate must be enabled on the API server

◆ PAUSE AND DECIDE

What is the cause, and the fix?

A Ingress was removed; rewrite everything using the Gateway API

The Ingress API is still stable. Only this old version went.

B That beta version was removed; use networking.k8s.io/v1

The kind lives on; the manifest names a removed beta version.

C The Ingress CRD is missing and needs to be installed

Ingress is built in, not a CRD.

D An alpha feature gate must be enabled on the API server

The version was beta, and removed versions cannot be switched back on.

THE SIGNAL

'No matches for kind X in version Y' while other X objects work: the version was removed, not the kind.

> Quayside wants developers to declare a FerryRoute object with kubectl. For each route, the cluster should create and maintain the right Deployments and Services.
>
> The team wants the standard, least-effort approach, with no custom API server to run.

What should Quayside build?

A An aggregated extension API server for ferry routes

B ConfigMaps holding routes, plus a script run by a CronJob

C A CustomResourceDefinition for FerryRoute, and nothing else

D A FerryRoute CRD plus a custom controller: an operator

◆ PAUSE AND DECIDE

What should Quayside build?

A. An aggregated extension API server for ferry routes

Works, but it is the custom API server they said they do not want.

B. ConfigMaps holding routes, plus a script run by a CronJob

No real kind, no validation, and drift is only fixed on a schedule.

C. A CustomResourceDefinition for FerryRoute, and nothing else

Stores the routes, but nothing creates or maintains anything.

D. A FerryRoute CRD plus a custom controller: an operator

The CRD gives the kind; the controller reconciles it.

THE SIGNAL

'Declare it and have the cluster maintain it' needs both a CRD and a controller.

> A Quayside engineer is editing the booking API Deployment and wants to know which fields spec.strategy accepts, and what each means.
>
> They want the answer from the cluster itself, without leaving the terminal.

Which command gives exactly that?

A kubectl explain deployment.spec.strategy

B kubectl api-resources

C kubectl describe deployment booking-api

D kubectl get deployment booking-api -o yaml

◆ PAUSE AND DECIDE

Which command gives exactly that?

A kubectl explain deployment.spec.strategy

Documents each field and its type, from the cluster's own schema.

B kubectl api-resources

Lists kinds, not the fields inside one.

C kubectl describe deployment booking-api

Shows one live object and its events, not field documentation.

D kubectl get deployment booking-api -o yaml

Shows current values, not what the fields mean or allow.

THE SIGNAL

'What does this field mean' is explain. 'What is in this object' is get or describe.

Module 4 in five lines

- Every object is **apiVersion, kind, metadata, spec**; the system adds **status**.
- **Core group is plain v1**; everything else is group/version, served over REST.
- **Alpha may vanish, beta may change, stable stays** within Kubernetes 1.x.
- **Declarative apply** plus level-triggered controllers corrects drift automatically.
- **CRD adds a noun, controller adds a verb**: together, an operator.

All of this state lives in one place. Next: **etcd, high availability** and the control plane at work.